
UCI HTTP Target

Command Summary

Gideon Zweijtzer

All work Copyright © 2026 by
Gideon's Logic B.V.
All rights reserved.

Table of Contents

1. Introduction	3
1.1. Context	3
1.2. Purpose of this document	3
2. Concept	4
2.1. Background HTTP: a Quick Summary	4
2.2. Mapping HTTP onto the UCI concept	5
3. Commands	6
3.1. HTTP_CMD_IDENTIFY (0x01)	6
3.2. HTTP_CMD_HEADER_CREATE (0x11)	6
3.3. HTTP_CMD_HEADER_FREE (0x12)	7
3.4. HTTP_CMD_HEADER_ADD (0x13)	7
3.5. HTTP_CMD_HEADER_QUERY (0x14)	7
3.6. HTTP_CMD_HEADER_LIST (0x15)	7
3.7. HTTP_CMD_BODY_CREATE (0x21)	7
3.8. HTTP_CMD_BODY_FREE (0x22)	8
3.9. HTTP_CMD_BODY_ADD_INT (0x23)	8
3.10. HTTP_CMD_BODY_ADD_BOOL (0x24)	8
3.11. HTTP_CMD_BODY_ADD_STRING (0x25)	9
3.12. HTTP_CMD_BODY_ADD_OBJECT (0x26)	9
3.13. HTTP_CMD_BODY_ADD_ARRAY (0x27)	10
3.14. HTTP_CMD_BODY_UP (0x28)	10
3.15. HTTP_CMD_BODY_REMOVE (0x29)	11
3.16. HTTP_CMD_BODY_QUERY (0x2A)	11
3.17. HTTP_CMD_BODY_MOVE (0x2B)	12
3.18. HTTP_CMD_BODY_ADD_BINARY (0x2C)	12
3.19. HTTP_CMD_DO_EXCHANGE_OBJ (0x31)	12
3.20. HTTP_CMD_DO_EXCHANGE_RAW (0x32)	13

1. Introduction

1.1. Context

The “HTTP Target” facilitates performing HTTP requests over the network, to offload the C64 from constructing the network packets and interpreting the network responses. The “HTTP Target” thus carries the responsibility to adhere to the HTTP protocol, at least to some degree.

The “HTTP Target” is a target of the “Ultimate Command Interface” (UCI), and is thus accessible from the cartridge port, through some I/O registers. The document “Ultimate Command Interface – Register API” describes how commands are sent over this interface.

1.2. Purpose of this document

This document describes the concept of this abstraction layer and the commands that can be sent to this target, and their expected behavior and response.

2. Concept

2.1. Background HTTP: a Quick Summary

HTTP is an omnipresent underlying protocol used for accessing resources on the world wide web. In the basis, before life became hard, it is a request-response protocol.

HTTP requests consist of a header section and possibly a body section. The header contains information about the request, like the destination (sub-)URL, the version of the protocol, and what 'method' is being used to access the resource. This method is called the 'verb'. Examples of the most common verbs are:

- GET (Safe/Idempotent): Retrieves data without modifying it.
- POST (Not Idempotent): Submits data to create a new, subordinate resource.
- PUT (Idempotent): Replaces a resource completely or creates it if it doesn't exist.
- PATCH (Not Idempotent): Applies partial modifications to a resource.
- DELETE (Idempotent): Removes a specific resource.

The header is basically a list of key-value pairs, separated by a colon, such as:

Transfer-Encoding: chunked

The GET and DELETE requests usually do not make use of the data body, while PUT, POST and PATCH usually do to transfer the data; also named 'form data'. When data is limited, it is sometimes encoded in the URL.

The format of the data body is not prescribed in the HTTP standard and can thus be of any format. However, what format is used is described in the header with the following key-value pair. In this case, it tells the receiving application that the data represents JSON formatted data fields.

Content-Type: application/json

Responses are formatted in a very similar way; with a header and a data body. The data body is obviously used for the GET requests, but it can also be used for other requests to encode application specific responses and error codes. The header always contains a response code. HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes, defined in RFC 9110:

- Informational responses (100 – 199)
- Successful responses (200 – 299)
- Redirection messages (300 – 399)
- Client error responses (400 – 499)
- Server error responses (500 – 599)

Very well known response codes are 200, "OK", and of course the famous 404, which means that the resource was not found (invalid URL).

So, we have established that for a single interaction with a server, there are four chunks of data: 1) the request header, 2) the request data body, 3) the response header, 4) response data body.

Let's see how the Ultimate application can assist with handling these.

2.2. Mapping HTTP onto the UCI concept

The Ultimate Command Interface is an interface that allows the exchange of commands and command responses through a few registers in the I/O space; often \$DF1B-\$DF1F. When the Ultimate application is ready to receive a command, the bytes of this command can be passed through a register. The hardware writes these bytes in the command buffer. When the command has been transferred, it is submitted to the command processor. The result is written into the response and status buffers and the C64 is ought to read these and acknowledge reception.

The limitation of this interface is that commands *and* their responses are atomic; only one can be in flight at any given time, as the command-response sequence follows a strict state machine. Because requests to a server may take several seconds to complete, this is something that must be taken into account. However, since the platform we're working on is a C64, this limitation is likely very acceptable.

Often, requests to a server follow a certain pattern that contain a lot of the same data; aka 'boiler plate'. To minimize the overhead, the UCI 'HTTP Target' provides 'handles' to the different chunks of data. By allowing the user to control creation, use and disposal of these handles, they can be modified and reused for maximum efficiency. Think of it as 'file handles' that can be opened, read/written and closed.

As previously stated, HTTP data falls into two categories: headers and data bodies. Therefore, in this concept these are handled separately. In order to send a request and receive a response, a header and an optional data body are created. When the request returns, new header and data body objects are created, which can then be read or queried.

Note that the use of header and data objects is also fully compatible with offloading the creation and parsing of valid JSON (or XML??) data bodies. Yeah, sometimes objectifying is a good thing 😊

3. Commands

Starting from version 3.15 the firmware of the Ultimate products and the Commodore 64 Ultimate, the “HTTP target” is accessible through target \$06. This shall be the first byte of the command.

The following paragraphs describe each of the commands of “HTTP target”.

To avoid repetition, all commands can return the status 400 BAD COMMAND, if the format of the command is not followed, or parameters are out of bounds.

3.1. HTTP_CMD_IDENTIFY (0x01)

Command format: \$06 \$01

The “Identify” command sends back an identification string, such as “ULTIMATE HTTP TARGET V1.0”. The user software can use this function to query which targets exist, or to obtain version information.

The status channel will report 000 OK, as this command cannot fail.

3.2. HTTP_CMD_HEADER_CREATE (0x11)

Command format: \$06 \$11 <VERB> <URL>

The “Header Create” command is used to create a new header object.

The verb byte is one of the following:

Verb	Value
GET	\$01
PUT	\$02
POST	\$03
PATCH	\$04
DELETE	\$05
HEAD	\$06
OPTIONS	\$07
CONNECT	\$08
TRACE	\$09

The URL string is the full resource specifier, including the hostname. This does two things. It sets the hostname associated with the header, so it does not need to be specified for every exchange, and it sets the sub-URL in the header. Example:

\$06 \$11 \$01 “commoserve.files.commodore.net/leet/search/bin” will store “commoserve.files.commodore.net” as the host to connect to, and set the initial header to:

```
GET /leet/search/bin HTTP/1.1
```

This command has only two possible outcomes; either there is a free header slot available, resulting in a status 000 OK, or when the resources have run out: 507 NO HEADER SLOT.

When the command is successful, it returns one single byte in the data channel, which is the handle of the header.

3.3. HTTP_CMD_HEADER_FREE (0x12)

Command format: \$06 \$12 <HANDLE>

This command is used to free the header resource.

This command will never fail; not even if the resource was already free. It always responds with 000 OK.

3.4. HTTP_CMD_HEADER_ADD (0x13)

Command format: \$06 \$13 <HANDLE> <HEADER LINE>

The “Header Add” command is used to fill the reusable header structure with a key value pair. The Header line is just one argument, so both the key as the value are passed as one string, separated by a colon and space. These must be present. When the key is already present in the header, it is replaced with the new one, so this command can also be used to modify an existing header entry.

If the header line is in the right format, the command returns with “000 OK”. If the format is wrong, the result is 500 BAD FORMAT.

This command does not return data.

3.5. HTTP_CMD_HEADER_QUERY (0x14)

Command format \$06 \$14 <HANDLE> <KEY>

This command is used to query the header. While this may not be very useful for the request header, it is useful to examine the response header.

When the key is present in the header, the data channel will return the value associated with the key-value pair. In this case the status channel is set to 000 OK. When the key is not present in the header, the status channel is set to 404 KEY NOT PRESENT.

3.6. HTTP_CMD_HEADER_LIST (0x15)

Command format \$06 \$15 <HANDLE> <INDEX>

This command is useful when the application does not know what key value pairs are in the header. The index is used to facilitate the processing of the data that is returned. Use \$00 if the entire header is requested, use a line/entry number starting with ‘1’ to get each key-value pair separately. When the command succeeds, the status channel will read 000 OK, and the data channel will contain the requested (part of) the header.

When the entry number exceeds the number of key-value entries, the status channel will return 400 INDEX OUT OF BOUNDS. In this case the data channel will be empty.

3.7. HTTP_CMD_BODY_CREATE (0x21)

Command format: \$06 \$21 <FORMAT>

The “Body Create” command is used to create a new data object for the HTTP body.

The format byte is one of the following:

Verb	Value
JSON	\$01
BINARY	\$02
URL ENCODED	\$03

This command has only two possible outcomes; either there is a free data slot available, resulting in a status `000 OK`, or when the resources have run out: `507 NO DATA SLOT`.

When the command is successful, it returns one single byte in the data channel, which is the handle of the body object.

In case the data object is of the 'JSON' or 'URL ENCODED' type, it automatically opens the top level object, which could be represented as `{ }`. This JSON body has a pointer associated to it, which can be moved to add items at the correct level of the hierarchy. When the body type is 'URL ENCODED', the formatting of the body will be different, but the internal data structure is the same.

3.8. HTTP_CMD_BODY_FREE (0x22)

Command format: `$06 $22 <HANDLE>`

This command is used to free the body data resource.

This command will never fail; not even if the resource was already free. It always responds with `000 OK`.

3.9. HTTP_CMD_BODY_ADD_INT (0x23)

Command format: `$06 $23 <HANDLE> <KEYLEN> <KEY> <INT>`

Use this command to add a primitive of type 'int' to the data object, in the current object hierarchy. Right after the command `HTTP_CMD_BODY_CREATE`, the current hierarchy level will be set to 'top'. Since key names are variable in length, the command starts with a one-byte length indicator of the string. Then the string itself follows, exactly the number of bytes indicated. Then the integer value follows, which can be 1, 2, 3 or 4 bytes in length. It will be interpreted as a signed value, LSB first.

Example: `"width": 500` would be encoded as: `$06 $23 $XX $05 "width" $F4 $01`. This results in the object:

```
{
  "width" : 500
}
```

In URL encoded form, the body would look like this:

```
width=500
```

3.10. HTTP_CMD_BODY_ADD_BOOL (0x24)

Command format: `$06 $24 <HANDLE> <KEYLEN> <KEY> <BOOL>`

Use this command to add a primitive of type 'bool' to the data object, in the current object hierarchy. Since key names are variable in length, the command starts with a one-byte length indicator of the

string. Then the string itself follows, exactly the number of bytes indicated. Then the value of the Boolean follows, always 1 byte in length.

Example: `"visible": true` would be encoded as: `$06 $24 $XX $07 "visible" $01`. This results in the object, if preceded by the previous example:

```
{
  "width" : 500,
  "visible" : true
}
```

In URL encoded form, the body would look like this:

```
width=500&visible=true
```

3.11. HTTP_CMD_BODY_ADD_STRING (0x25)

Command format: `$06 $25 <HANDLE> <KEYLEN> <KEY> <VALUELEN> <VALUE>`

Use this command to add a primitive of type 'string' to the data object, in the current object hierarchy. Since key names are variable in length, the command starts with a one-byte length indicator of the string. Then the string itself follows, exactly the number of bytes indicated. Then the value of the string follows, once again encoded with a length byte first.

Example: `"title": "Commodore Intl"` would be encoded as:

`$06 $25 $XX $05 "title" $0E "Commodore Intl"`.

This results in the object, if preceded by the previous examples:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore Intl"
}
```

In URL encoded form, the body would look like this:

```
width=500&visible=true&title=Commodore%20Intl
```

3.12. HTTP_CMD_BODY_ADD_OBJECT (0x26)

Command format: `$06 $26 <HANDLE> <KEYLEN> <KEY>`

Use this command to add a new sub-object to the current data object, in the current object hierarchy. Since key names are variable in length, the command starts with a one-byte length indicator of the string. Then the string itself follows, exactly the number of bytes indicated. This command does not take any value.

Example:

`$06 $26 $XX $04 "user"` results in the following object, if preceded by the previous examples:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore",
  "user" : { ■ }
}
```

After this command, the 'current hierarchy level' will be at the position of the black square.

In URL encoded form, the body would look like this:

```
width=500&visible=true&title=Commodore%20Int1&user=%7B%7D
```

3.13. HTTP_CMD_BODY_ADD_ARRAY (0x27)

Command format: \$06 \$27 <HANDLE> <KEYLEN> <KEY>

Use this command to add a new array to the current data object, in the current object hierarchy. Since key names are variable in length, the command starts with a one-byte length indicator of the string. Then the string itself follows, exactly the number of bytes indicated. This command does not take any value.

Example:

\$06 \$27 \$XX \$04 "cars" results in the following object, if preceded by the previous examples:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore",
  "user" : {
    "cars" : [ ■ ]
  }
}
```

After this command, the 'current hierarchy level' will be at the position of the black square.

In URL encoded form, the body would look like this:

```
width=500&visible=true&title=Commodore%20Int1&user=%7Bcars%3D%5B%5D%7D
```

3.14. HTTP_CMD_BODY_UP (0x28)

Command format: \$06 \$28 <HANDLE>

The "Up" command is used to step one level up the hierarchy, or in other words it ends the addition of items in the current level. The command does not take any arguments. Note that it is not required to step all the way up to the root for the data object to be valid for use. Repeated calls to the "Up" command can be used to walk to the root. No error is generated when the pointer is already in the root of the hierarchy.

Example:

\$06 \$28 \$XX results in the following object, if preceded by the previous examples:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore",
  "user" : {
    "cars" : [ ]
    ■
  }
}
```

After this command, the 'current hierarchy level' will be at the position of the black square.

3.15. HTTP_CMD_BODY_REMOVE (0x29)

Command format: \$06 \$29 <HANDLE> <PATH>

Use the "Remove" command to delete one particular entry from the object. The path is a string that leads to the entry of interest. Let's say the data object in memory looks like this:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore",
  "user" : {
    "name" : "Peri"
  }
}
```

Example: \$06 \$29 \$XX "user/name" will result the key/value pair "name: Peri" to be removed from the object. \$06 \$29 \$XX "user" will remove the entire object "user", including all child objects.

The status channel will read 000 OK when the path is valid. When the handle is valid, but the path does not lead to a known value, the status will be set to 404 ENTRY NOT FOUND. In case the handle is invalid, the status will be set to 400 BAD REQUEST.

The command does not return data.

3.16. HTTP_CMD_BODY_QUERY (0x2A)

Command format: \$06 \$2A <HANDLE> <PATH>

Use the Query command to read the value of an entry in the data body. The path is a string that leads to the entry of interest. The return value starts with a one-byte type identifier:

Type	Value
Integer	\$01
Boolean	\$02
String	\$03
Object	\$04
Array	\$05

Let's say the object looks like this:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore",
  "user" : {
    "name" : "Peri"
  }
}
```

Example: `$06 $2A $XX "user/name"` will result in a data message `$03 "Peri"`, without the quotes.

The status channel will read `000 OK` when the path is valid and the value is returned. When the handle is valid, but the path does not lead to a known value, the status will be set to `404 ENTRY NOT FOUND`. In case the handle is invalid, the status will be set to `400 BAD REQUEST`.

3.17. HTTP_CMD_BODY_MOVE (0x2B)

Command format: `$06 $2B <HANDLE> <PATH>`

Use the Move command to position the cursor somewhere in the data object hierarchy, in order to add (or set) values. The path is a string that leads to the entry of interest. Let's say the object looks like this:

```
{
  "width" : 500,
  "visible" : true,
  "title" : "Commodore",
  "user" : {
    "name" : "Peri"
  }
}
```

Example: `$06 $2B $XX "user"` will make the cursor move into the object "user", alongside the key "name".

The status channel will read `000 OK` when the path is valid and points to an object or to an array. When the handle is valid, but the path does not lead to an object or an array, the status will be set to `404 ENTRY NOT FOUND`. In case the handle is invalid, the status will be set to `400 BAD REQUEST`.

3.18. HTTP_CMD_BODY_ADD_BINARY (0x2C)

Command format: `$06 $2C <HANDLE> <Binary Data>`

The command can be used to add binary data to body handle that is of the binary type. When the handle points to another type, the status channel returns `400 BAD REQUEST`. Multiple additions of binary data can be done when the size of the data exceeds what can be sent in one command. The maximum command size to the UCI is 896 bytes.

3.19. HTTP_CMD_DO_EXCHANGE_OBJ (0x31)

Command format: `$06 $31 <HEADER> <BODY>`

This command performs the HTTP exchange on the network. It references the URL and header with the first byte, and the optional data body with the second byte. When no body is to be sent, the value `$00` can be used.

The status channel will represent the information given in the header of the response, including the response code. If no connection can be made, the status channel reads `503 SERVICE UNAVAILABLE`.

When the exchange succeeds, it creates two new objects; a header object and a body object. The handles of these objects are returned in the data channel (2 bytes).

3.20. HTTP_CMD_DO_EXCHANGE_RAW (0x32)

Command format: `$06 $32 <HEADER> <BODY>`

This command performs the HTTP exchange on the network. It references the URL and header with the first byte, and the optional data body with the second byte. When no body is to be sent, the value `$00` can be used.

The status channel will contain the entire response header, limited to 256 bytes. If no connection can be made, the status channel reads `503 SERVICE UNAVAILABLE`.

When the exchange succeeds, the data channel will produce the body data “as is”. This might need to be read in multiple chunks when the UCI indicates the “more data” flag. No processing on the data is done.